

ACADEMIC PROCESS COPILOT

A bilingual assistant that answers university-regulation questions
only from verified records — and says so when it cannot

Project Report

Portfolio project for an AI-Enabled Academic and Administrative Support IT Technician application

Safa Mohammed Al Shukaili

B.Tech Software Engineering, University of Technology and Applied Sciences (UTAS) – Ibra
September 2026

github.com/Safa-Alshukaili/academic-process-copilot

Contents

Summary	1
1. Problem	1
2. Objectives and scope	1
3. Data source and database design	2
4. System architecture	2
5. Understanding the question	3
5.1 Normalization, stemming and a domain synonym table	3
5.2 Confidence gate	4
5.3 Showing what was understood	4
6. Preventing invented answers	4
7. Closing the loop: unanswered questions and staff notification	5
8. Evaluation	7
8.1 Method	7
8.2 Results	7
8.3 Remaining errors	8
9. Testing and quality assurance	8
10. Problems found during development	9
11. Responsible use, privacy and security	10
12. Deployment and operation	10
13. Limitations and future work	10
14. Conclusion	11
15. Development notes	11
References	11
Appendix A. Reproducing the results	12

Summary

Academic Process Copilot answers students' questions about university procedures — course load, withdrawal, attendance, deferral, probation, training, graduation and others — in English or Arabic. Its knowledge is a small SQLite database of 14 procedures and 28 question-and-answer entries written from the UTAS Academic Regulation (Decision No. 612/2022), each citing the article it comes from.

The design goal is narrow and testable: **every sentence a student reads must come from that database**. The system does not generate answer text. It picks one verified record, shows it word-for-word together with the question it matched, and refuses when no record covers the question well enough. Refused questions are logged, similar phrasings are grouped, and staff receive a scheduled email listing what students asked that the data could not answer, so the gaps can be closed.

On a held-out set of 34 questions written before any tuning, the current version answered 19 of 22 in-scope questions from exactly the right entry and none from an unrelated topic, refused 2 answerable questions, and wrongly answered 2 of 12 out-of-scope questions (Section 8). These are small test sets written during development, not questions from real students; they indicate direction, not production accuracy. The project is exposed as a FastAPI service and a command-line demo, runs in Docker, is tested automatically on every push with GitHub Actions, and uses two n8n workflows for student inquiries and the staff email digest.

1. Problem

Registration offices, academic advisors and IT support staff answer the same regulation questions many times each semester: how many credit hours are allowed, the last week to withdraw, what GPA leads to probation. The answers are fixed and published, so answering them by hand is repetitive work. An automated assistant can help, but only if it is reliable: a wrong deadline or GPA threshold given confidently to a student is worse than no answer, because the student may act on it.

General-purpose chatbots are a poor fit for this. They can produce fluent text that is not in any regulation — often called hallucination [4] — and they rarely say "this is not covered". The question this project explores is therefore not "can an AI answer regulation questions?" but "can a small system answer only what the regulation actually says, show where it came from, and hand everything else to a person?"

2. Objectives and scope

- Answer procedure questions in the language asked (English or Arabic), from verified and article-cited data only.
- Refuse questions the data does not cover, instead of answering from the nearest-looking entry.
- Make every answer checkable: show the matched question, the source article and the office to verify with.

- Close the loop: log unanswered questions, group repeated ones, and notify staff automatically.
- Deliver it as a documented, containerized, automatically tested service, plus no-code automation.

Out of scope: the full 92-article regulation (the database covers the most commonly asked topics), authentication, an admin user interface, and any use with real student data.

3. Data source and database design

All institutional content comes from the UTAS Academic Regulation, Decision No. 612/2022, published in the Official Gazette, Issue 1468 [1]. The data was curated into question-and-answer form rather than loaded as raw legal text, because students ask specific questions ("until when can I withdraw?") that map to specific clauses.

Table 1. Database tables (SQLite, built by `data/seed.py`)

Table	Purpose	Key fields
processes	14 procedures (registration, withdrawal, deferral, ...)	name, description, responsible office — English and Arabic
process_steps	Ordered steps per procedure	step number, title, description, required documents
forms	Forms linked to a procedure	form name, where to obtain it
faqs	28 verified question/answer pairs	question, answer (EN + AR), process_id, last_verified_date, verified_by
audit_log	Every question asked	timestamp, question, matched ids, QA result, language — no answer text, no personal data

Every user-facing field exists separately in English and Arabic. A question is scored only against the fields of its own language, so an Arabic question cannot be matched through English text, and the answer — including labels such as "Verify with" — is rendered entirely in that language. Each FAQ records when and by whom it was last verified; `GET /admin/freshness` lists entries not re-verified within 180 days.

4. System architecture

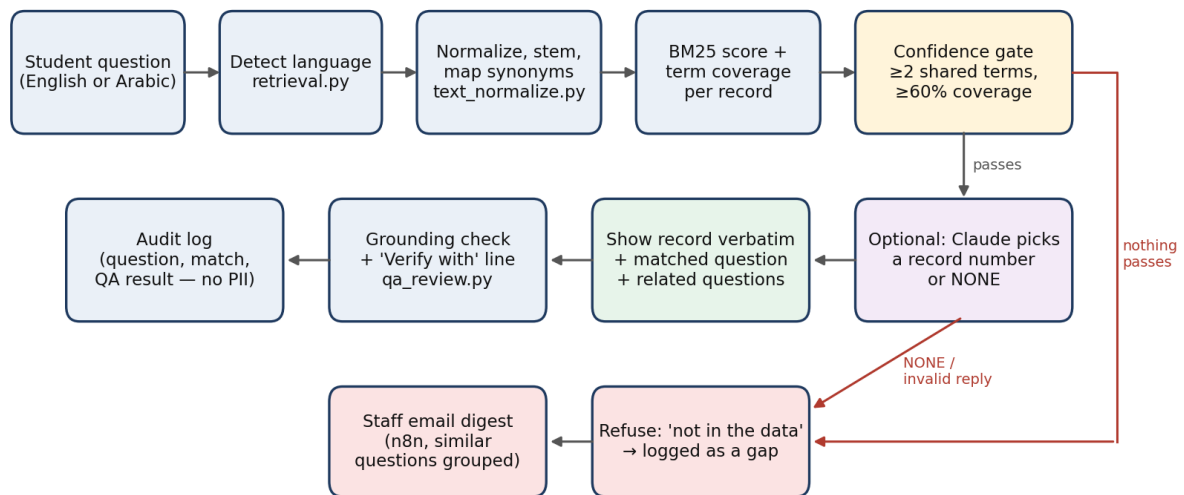


Figure 1. Question-answering pipeline. Red paths are refusals: they are logged and reach the staff email digest.

Table 2. Main components

Component	Responsibility
src/text_normalize.py	Arabic/English normalization, light stemming, citation removal, domain synonym table
src/retrieval.py	Language detection; BM25 scoring [2][6] plus term-coverage signals for every record
src/agent.py	Confidence gate, optional LLM selection, verbatim rendering, logging
src/qa_review.py	Automated checks: verify line present, context found, numbers and article citations grounded
src/audit_log.py, src/gap_grouping.py	Logging, KPIs, unanswered-question queue, grouping of similar phrasings
src/api.py	FastAPI service [7]: /ask, /health, /admin/unanswered, /admin/unanswered/grouped, /admin/kpi, /admin/freshness
automation/*.n8n.json	n8n workflows [8]: student inquiry (webhook) and staff email digest (schedule)

5. Understanding the question

The first version used BM25 ranking over the question and answer text [2]. BM25 matches identical words only, which caused two opposite failures found during testing:

- **Answerable questions refused.** An Arabic question asking when a student is barred from the final exam returned "not found", although FAQ 9 answers it: the FAQ and the student used different forms of the same Arabic word (the question and today's answer are shown in Section 5.3). Colloquial synonyms, such as "highest" instead of "maximum", failed the same way.
- **Unanswerable questions answered from the wrong entry.** "How many credit hours do I need before applying for OJT?" shares "credit" and "hours" with course-load FAQs and was

answered with the 21-credit-hour rule — a correct regulation for a different question (Figure 2). The regulation sets no credit-hour threshold for training, so the right answer is a refusal.

```
PS C:\Users\ssaf\Downloads\academic-process-copilot\academic-process-copilot> Invoke-RestMethod -Uri "http://localhost:5678/webhook-test/student-inquiry" -Method POST -ContentType "application/json" -Body '{"question": "How many credit hours do I need for OJT?"}'

status  answer
-----
answered Yes, up to 21 credit hours, if you achieved at least a 3.50 semester GPA over 15+ hours in...

PS C:\Users\ssaf\Downloads\academic-process-copilot\academic-process-copilot>
```

Figure 2. Before the confidence gate: the OJT question was answered from an unrelated course-load FAQ (live call through the n8n webhook).

5.1 Normalization, stemming and a domain synonym table

`text_normalize.py` applies the same steps to questions and to every stored record: it unifies Arabic spelling variants (the forms of alef, ta marbuta and alef maqsura; diacritics and Arabic punctuation removed), strips common prefixes and suffixes (the definite article, attached conjunctions and prepositions, plural and possessive endings; -s, -ed and -ing in English) in the spirit of Arabic light stemming [5], removes article citations before scoring — otherwise the Arabic word for "article", which students also use to mean "course", matched every answer — and maps words with the same meaning in this domain to one concept (for example, several Arabic forms of "barred from the exam", "postpone / defer", and the two Arabic words for "course").

The synonym table is written by hand. It is small, transparent and easy to review, but it only covers words someone thought of; adding FAQs on a new topic may require new entries. A multilingual embedding model could generalize better, at the cost of a large dependency and a model download; it was not added, and no claim is made about how it would perform.

5.2 Confidence gate

For each record, retrieval now reports not only a BM25 score but how many of the question's distinct terms it contains and what fraction (coverage). Words that appear nowhere in the database still count in the denominator, so a question about "library late fees" cannot look fully covered. A record may answer only if it shares at least two terms and covers at least 60% of the question; otherwise the system refuses. The thresholds were chosen on the development question set only (Section 8). Raising them reduces wrong answers and increases refusals — a trade-off that should be revisited with real student questions.

5.3 Showing what was understood

Every answer starts with the verified question it was matched to, and ends with the other questions in the same procedure. The most common remaining error is choosing a sibling entry in the right procedure (for example, the withdrawal deadline when the student asked how many withdrawals are allowed). With the matched question visible and the siblings listed, that error is visible to the student and easy to recover from. Real output:

Q: متى يصل الطالب للحرمان؟
 أقرب سؤال موثوق: كم إنذار غياب أحصل قبل ما أحرم من الاختبار النهائي؟
 إنذار أول عند نسبة غياب 10%، وإنذار ثانٍ عند 15%، والحرمان من دخول الاختبار النهائي (واعتباره راسيًا) عند تجاوز الغياب 20% في المقرر (المادة 58).
 أسئلة ذات صلة يمكنك طرحها:

- وش يصير لو غبت أكثر من 20% من ساعات المقرر؟
تحقق مع: القسم الأكاديمي

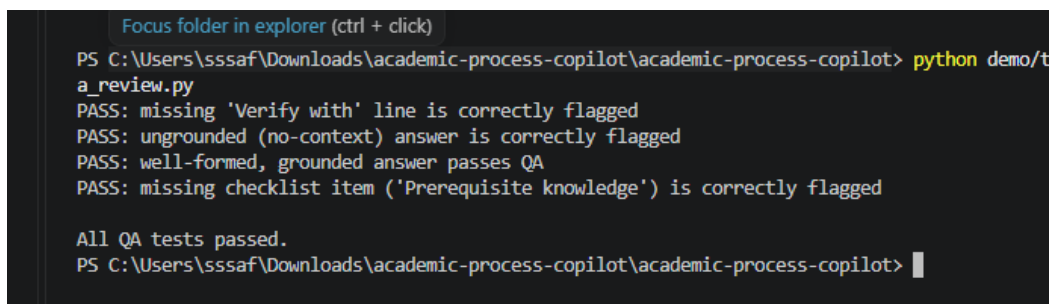
Q: How many credit hours do I need before applying for OJT?
I couldn't find this in the institutional database. Please contact the Registrar directly.

6. Preventing invented answers

Retrieval-augmented generation (RAG) normally retrieves documents and asks a language model to write an answer from them [3]. Even with correct retrieval, the model can paraphrase in ways that add or change facts [4]. This project removes free-text generation from the answer path:

1. **Verbatim answers.** The student always sees a stored record, copied exactly, with its article citation.
2. **Selection-only LLM (optional).** When `LLM_PROVIDER=anthropic` is set, Claude receives the question and up to three records that passed the gate and must reply with a record number or NONE. Any other reply — an explanation, an attempted answer, an out-of-range number — is treated as NONE. `demo/test_llm_selection.py` checks this with a stubbed model that tries to answer in prose; its text never reaches the student. The model adds a second, stricter judgment of which record fits; it never adds content.
3. **Grounding check.** `qa_review.check_grounding()` extracts every number and article citation from the answer and verifies each appears in the retrieved context. With verbatim answers this is a second line of defense; a test feeds it a fabricated answer (wrong GPA, invented "Article 99") and confirms it is caught (Figure 3).

What remains is choosing the wrong record — a real regulation that answers a different question. That risk is measured, not assumed away (Section 8).



```

Focus folder in explorer (ctrl + click)
PS C:\Users\ssaf\Downloads\academic-process-copilot\academic-process-copilot> python demo/t
a_review.py
PASS: missing 'Verify with' line is correctly flagged
PASS: ungrounded (no-context) answer is correctly flagged
PASS: well-formed, grounded answer passes QA
PASS: missing checklist item ('Prerequisite knowledge') is correctly flagged

All QA tests passed.
PS C:\Users\ssaf\Downloads\academic-process-copilot\academic-process-copilot>

```

Figure 3. QA tests: a missing "Verify with" line, an answer with no context and a missing checklist item are all flagged; a well-formed answer passes.

7. Closing the loop: unanswered questions and staff notification

Every refusal fails QA and is written to the audit log. `GET /admin/unanswered` lists these questions by frequency. Because students phrase the same gap differently, `GET /admin/unanswered/grouped` merges similar phrasings (Arabic and English normalized, lexical similarity) and keeps only groups asked at least twice, so one-off questions do not create noise.

Two n8n workflows use the API without application code. The student-inquiry workflow receives a question by webhook, calls `/ask`, and either returns the answer or escalates (Figures 4–5). The digest workflow runs on a schedule, calls the grouped endpoint, builds an HTML email (subject: the most-asked gap; body: a table of groups with their variants, counts and last-asked time), and sends it through Gmail SMTP with an app password [10]; it stays silent when there is nothing to report (Figure 8). Staff then add a verified answer with `src/add_faq.py`, which — unlike re-running `data/seed.py` — keeps the audit history.

```
PS C:\Users\ssaf\Downloads\academic-process-copilot\academic-process-copilot> Invoke-RestMethod -Uri "http://localhost:5678/webhook-test/student-inquiry" -Method POST -ContentType "application/json" -Body '{"question": "what is the weather today"}'

status    message
-----
escalated This question could not be answered from verified data and has been forw...
```

Figure 4. Student-inquiry workflow escalating an out-of-scope question ("what is the weather today") instead of guessing.

```
PS C:\Users\ssaf\Downloads\academic-process-copilot\academic-process-copilot> Invoke-RestMethod -Uri "http://localhost:5678/webhook-test/student-inquiry" -Method POST -ContentType "application/json" -Body '{"question": "What is the library late fee for overdue books?"}'

status    message
-----
escalated This question could not be answered from verified data and has been forwarded to staff fo...

PS C:\Users\ssaf\Downloads\academic-process-copilot\academic-process-copilot>
```

Figure 5. Escalation of "What is the library late fee for overdue books?", which is not in the regulation data.

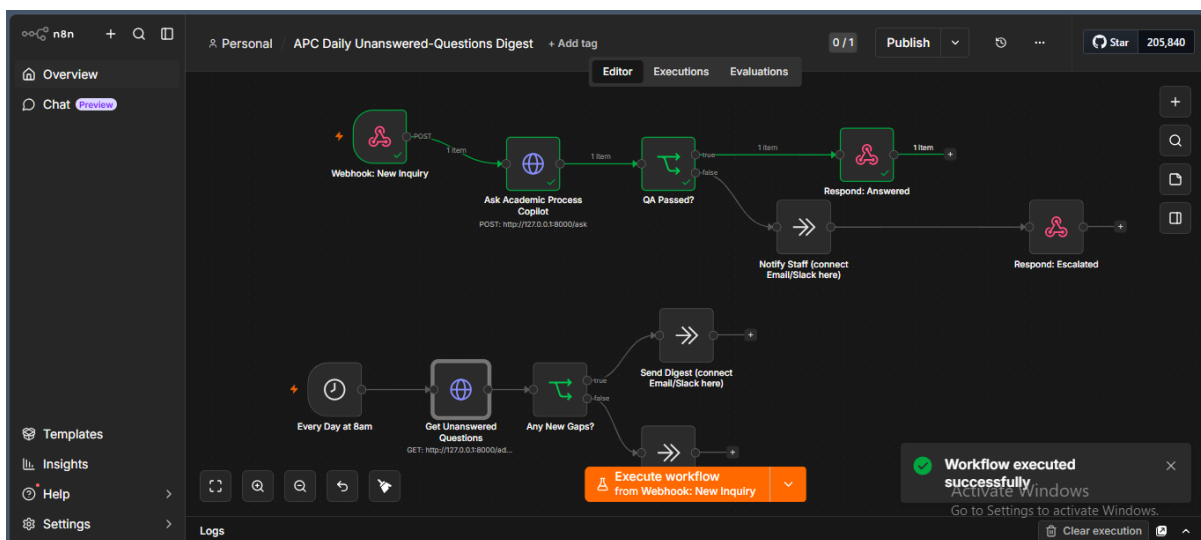


Figure 6. n8n canvas after a successful student-inquiry run (webhook → `/ask` → QA passed? → respond). The digest branch shown here is an earlier version with a placeholder notification node.

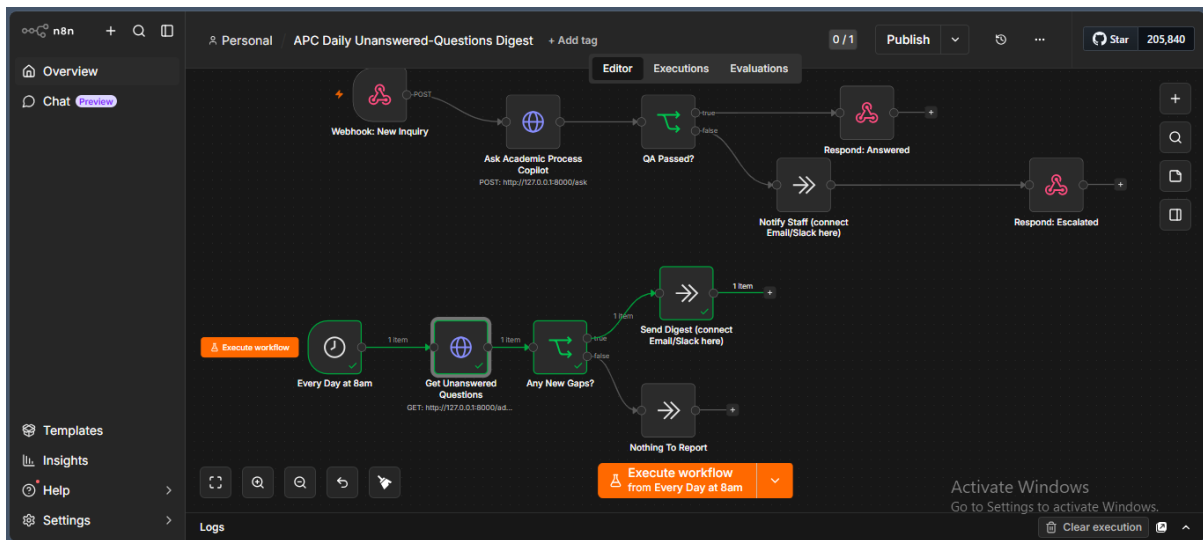


Figure 7. Scheduled digest run on the same canvas (earlier version, before the email step was added).

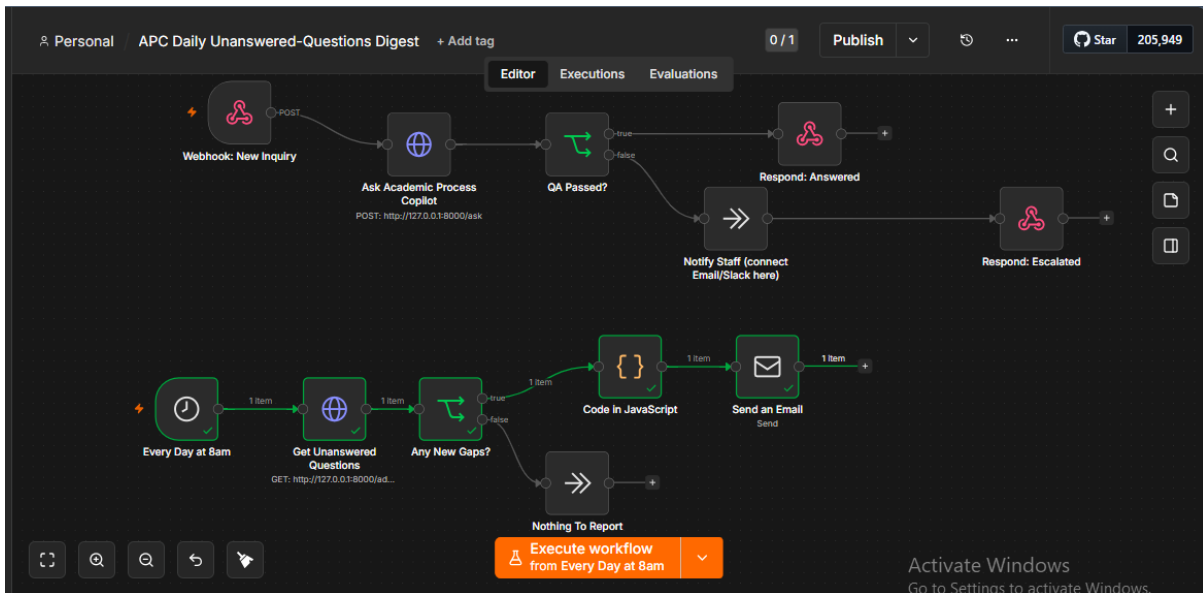


Figure 8. Scheduled digest run on the same canvas (last version, after the email step was added).



Figure 8. A digest email actually received: grouped unanswered questions, most-asked first, with the count and time last asked.

Operational constraint: the schedule only fires while n8n and the API are running. On a laptop, a closed lid means no email; unattended operation needs persistent hosting (n8n Cloud or a small server).

8. Evaluation

8.1 Method

`demo/eval_questions.py` holds labeled questions; each is tagged with the FAQ or procedure that answers it, or with "refuse". There are two sets, both written during development rather than collected from real students:

- **DEV** — 35 in-scope and 9 out-of-scope questions, used while tuning normalization, the synonym table and the gate. Results on it are optimistic by construction.
- **HELDOUT** — 22 in-scope and 12 out-of-scope questions, written before any tuning and not edited afterwards. This is the reported number.

`demo/eval_retrieval.py` runs every question through the same routing the API uses and counts: exact matches; answers from the right procedure but a sibling entry; answers from an unrelated procedure; refusals of answerable questions; and out-of-scope questions answered anyway. The same script was run on the previous version for comparison. CI fails if a change makes the held-out numbers worse.

8.2 Results

Table 3. Held-out results, before and after normalization and the confidence gate

HELDOUT (22 in-scope / 12 out-of-scope)	Before	After
Answered from exactly the right FAQ	17 / 22	19 / 22
Answered from the right procedure (incl. sibling FAQ)	18 / 22	20 / 22
Answered from an unrelated topic	1 / 22	0 / 22
Refused although answerable (becomes a staff gap)	3 / 22	2 / 22
Out-of-scope question answered instead of refused	1 / 12	2 / 12

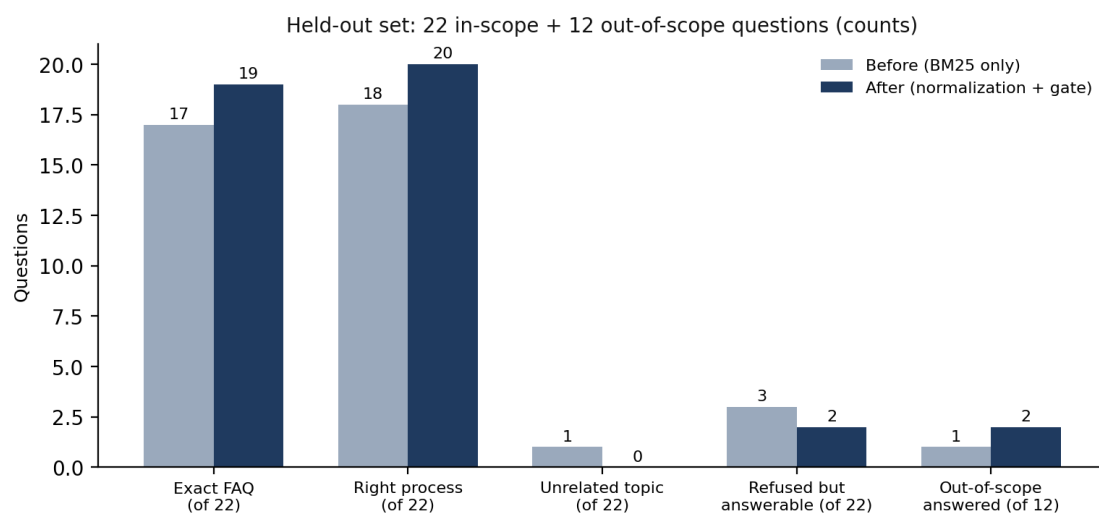


Figure 9. Held-out results as counts. Lower is better for the three right-hand groups.

On DEV, refusals of answerable questions fell from 11 of 35 to 0 and the OJT question is now refused; because DEV was used for tuning, that is an upper bound.

8.3 Remaining errors

- "Is there a bus between the branches?" (asked in Arabic) was answered with the branch-transfer procedure through the shared word "branches".
- "How many credit hours do I need to start my graduation project?" matched course-load rules, as in the previous version.
- "If my semester GPA was 3.6 can I register extra hours?" and "How many students must register for a course to open?" (asked in Arabic) were refused although answerable — they would reach staff as gaps, which is the safe failure.
- "Can I register 21 hours?" (asked in Arabic) was answered with the maximum-load entry instead of the exception entry — right procedure, sibling FAQ.

One out-of-scope answer more than before is the cost of accepting more paraphrases. With 12 out-of-scope questions, one question equals 8 percentage points, so the difference is within the noise of a set this small. The next evaluation should use real, anonymized student questions collected from the audit log.

9. Testing and quality assurance

Table 4. Automated tests (all run in GitHub Actions on every push [9])

Test	What it checks
<code>test_qa_review.py</code>	QA passes a well-formed answer and flags a fabricated one (invented number and article)
<code>test_bilingual.py</code>	Arabic and English do not cross-match; Arabic out-of-scope questions escalate in Arabic
<code>test_understanding.py</code>	The Arabic "barred from the exam" question is answered from FAQ 9; the OJT question is refused; citations are ignored; related questions listed
<code>test_llm_selection.py</code>	With a stubbed model: a chosen record is shown verbatim; NONE, prose or an invalid number all become refusals
<code>test_add_faq.py</code>	A gap can be closed without wiping the database
<code>test_audit_and_freshness.py</code> , <code>test_kpi.py</code>	Logging, stale-data detection and the aggregated health view
<code>test_gap_grouping.py</code>	Similar Arabic and English phrasings are merged; one-off questions are filtered out
<code>eval_retrieval.py --ci</code>	Held-out accuracy, wrong-topic and false-answer rates do not regress

Human review remains part of the process (`docs/QA_PROCESS.md`): the automated checks support a reviewer and never replace sign-off for new content.

10. Problems found during development

The following defects were found by testing the system, not anticipated in the design. Each has a fix and, where possible, a test.

Table 5. Defects found and fixed

#	Problem	How it was found	Fix
1	"late" matched inside "calculated"	Wrong FAQ for an unrelated query	Word-boundary matching, later BM25 tokenization
2	One rare shared word ("fee") made an unrelated record rank first	Library-fee question matched a procedure	Require ≥ 2 shared distinct terms
3	FAQ-only answers failed QA (no "Verify with" line)	Test of adding a FAQ without a procedure	Verify line taken from the FAQ's verified-by field
4	Correct procedure identified by the FAQ but not in the BM25 top results	Arabic testing	Fetch the linked procedure directly by id
5	CI never installed dependencies	Reviewing the workflow after adding <code>rank_bm25</code>	Explicit pip install step
6	Arabic paraphrases not understood ("barred" question)	Asking the question live	Normalization, stemming, synonym table (Section 5.1)
7	The Arabic word for "article" in every citation matched every answer	Inspecting token overlaps	Citations removed before scoring

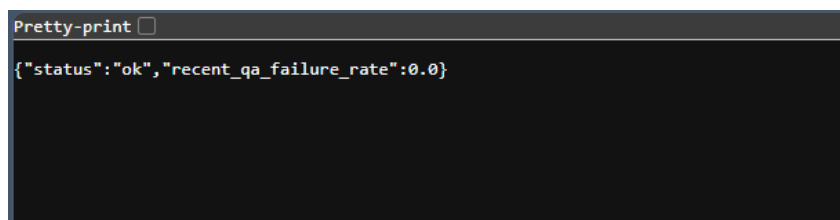
#	Problem	How it was found	Fix
8	Confident answers from the wrong regulation (OJT)	Live n8n test (Figure 2)	Confidence gate (Section 5.2)
9	Same gap reported once per phrasing	Reading the unanswered queue	Grouped endpoint for the digest
10	KPI test failed depending on test order	Running the suite in a different order	Test clears the audit log in its temporary copy
11	Personal email in an exported workflow and the database (with its audit log) committed to the public repository	Repository review	Placeholder address; database excluded via .gitignore and rebuilt by seed.py

11. Responsible use, privacy and security

- The audit log stores the question, what matched and the QA result — never the answer text or any personal identifier. Students are never asked for personal information.
- Stored records are treated as untrusted input to prompts: `security.py` strips prompt-injection patterns before any record is sent to a model.
- Answers always name the office to verify with; the system supports staff and does not replace their decisions.
- Credentials (API keys, the SMTP app password) live in environment variables or n8n's credential store, never in the repository.
- A real deployment would need authentication (the access check is a stub), institutional hosting, and review against Oman's Personal Data Protection Law (Royal Decree 6/2022) [12]. The NIST AI Risk Management Framework [11] is a useful reference for documenting risks such as the wrong-record errors in Section 8.

12. Deployment and operation

The service builds and runs with Docker (`docker build -t apc . && docker run -p 8000:8000 apc`); the image rebuilds the database from `data/seed.py`. Interactive API documentation is generated automatically at `/docs`. `GET /health` reports liveness and the recent QA failure rate (Figure 10); `GET /admin/kpi` combines question volume, pass rate, top gaps, stale entries and coverage in one view.



```

Pretty-print ☐
{"status": "ok", "recent_qa_failure_rate": 0.0}

```

Figure 10. Health endpoint responding live.

A browser-based demo (link in the README) needs no installation, but it is a separate copy that runs the earlier retrieval logic and does not log questions; the loop described in Section 7 runs only through the API.

13. Limitations and future work

- Understanding is lexical with a hand-written synonym table; unseen wording still fails. Candidate next step: a multilingual embedding model as an additional signal, evaluated on the same held-out set before adoption.
- valuation sets are small and were written during development, not collected from real students. Next step: an anonymized set of real student questions from the audit log.
- Answers are verbatim records — predictable but not conversational, and one answer cannot combine two entries.
- Coverage is a curated subset of the regulation, not all 92 articles.
- No authentication, no admin interface (data is added by script), and the n8n schedule needs persistent hosting to run unattended.
- The browser demo has not been updated to the gated retrieval, and the material-drafting module is English-only.

14. Conclusion

The project shows a way to use AI in an institutional setting where wrong answers carry real cost: restrict what the system may say to verified records, make the source of every answer visible, refuse when unsure, and route the refusals to people who can fix the data. The measured result is modest and honest — fewer wrong-topic answers and fewer missed questions on a small held-out set, with one additional false answer — and the evaluation, tests and documented limitations are there so that the next change can be judged by the same numbers.

15. Development notes

The project was developed with the help of AI coding assistants, used as a development tool under my direction. I chose the problem and the data source (the UTAS Academic Regulation) and set the requirements — including that answers must come only from verified data, with no generated content, and that repeated unanswered questions must reach staff by email. I tested the system end to end through the API, n8n and the email digest, and used that testing to drive changes: for example, support for Arabic paraphrases was added after a question about being barred from the exam returned "not found". Every change was tested before being kept. Behavior described in this report is backed by the code and automated tests in the repository and by screenshots of live runs; the figures in Section 8 are produced by `demo/eval_retrieval.py` and can be reproduced with one command.

References

- [1] University of Technology and Applied Sciences. (2022). Academic Regulation, Decision No. 612/2022. Official Gazette of the Sultanate of Oman, Issue 1468.

- [2] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [3] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33.
- [4] Ji, Z., Lee, N., Frieske, R., et al. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12).
- [5] Larkey, L. S., Ballesteros, L., & Connell, M. E. (2002). Improving stemming for Arabic information retrieval: Light stemming and co-occurrence analysis. *Proceedings of ACM SIGIR 2002*, 275–282.
- [6] Brown, D. rank_bm25: A collection of BM25 algorithms in Python. https://github.com/dorianbrown/rank_bm25
- [7] FastAPI documentation. <https://fastapi.tiangolo.com>
- [8] n8n documentation. <https://docs.n8n.io>
- [9] GitHub Actions documentation. <https://docs.github.com/actions>
- [10] Google Account Help. Sign in with app passwords. <https://support.google.com/accounts/answer/185833>
- [11] National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1.
- [12] Sultanate of Oman. (2022). Personal Data Protection Law, Royal Decree No. 6/2022.

Appendix A. Reproducing the results

```

pip install -r requirements.txt
python data/seed.py                # build the database
python demo/eval_retrieval.py      # Section 8 numbers
python demo/test_understanding.py  # Section 5 cases
python demo/test_llm_selection.py  # Section 6 guarantees
uvicorn src.api:app --reload        # API at http://127.0.0.1:8000/docs

```